

ITS THE 1 SOLUTIONS

ITSthe1.ID API

Integration Specification and Payload Reference

Build v1.0.0

Issued 5 August 2026 · for partner development teams

Companion to the 1.ID API Specification Document.

Where the two disagree, this document describes what the software actually does.

Contents

Contents	2
About this document	4
Who should read what	4
Base URL and transport	4
Request headers	4
The message envelope	4
How failures are reported	5
Identifiers: the one trap that costs the most time	5
Reference tables	6
Code sets	6
Escort types	6
Common structures	7
GuestInfo	7
AttachmentInfo	8
1. Checkin	9
1.1 Checkin GET	9
1.2 Checkin POST	9
1.3 Checkin PUT	16
1.4 Checkin AddBackdatedCheckin POST	18
2. Checkout	20
2.1 Checkout POST	20
2.2 Checkout PUT	21
3. Checkin Cancellation	22
3.1 CheckinCancellation POST	22
4. Guest	23
4.1 Guest GET	23
4.2 Guest POST — add a guest or escort	23
4.3 Guest PUT — amend a guest	25
5. Guest Checkout	27
5.1 GuestCheckout POST	27
6. Guest Attachment	29
6.1 GuestAttachment POST	29
7. Room Change	31
7.1 RoomChange POST	31
8. Reference lookup endpoints	33
Diagnostic table dumps	33

9. Monitoring endpoints	34
Appendix A. Deviations from the 1.ID specification	35
A.1 Error codes defined but not implemented	35
A.2 Codes returned that the specification does not define	36
A.3 Codes carrying more than one meaning	36
A.4 Validation rules that differ	36
A.5 Other differences	36
Appendix B. Known issues	37
Appendix C. Setting up a test environment	38
C.1 Prerequisites	38
C.2 Configuration	38
C.3 Starting the service and confirming it works	38
C.4 Accompanying files	38
C.5 A working call sequence	39
Appendix D. Security and data handling	40
Support	40

About this document

This document specifies every endpoint served by build v1.0.0 of the ITSthe1.ID API, together with a ready-to-send payload for each one. It follows the section numbering of the 1.ID API Specification Document so the two can be read side by side.

It differs from that document in one important way: **every statement here was verified against the source code of this build**. Where the specification and the software disagree, the difference is recorded in Appendix A rather than silently resolved.

Who should read what

If you are...	Read
Writing the client	Sections 1–7, then Appendix A
Setting up a test environment	Appendix C, then Section 9
Investigating a failure	Appendix B, then the validation order table for that endpoint
Reviewing the contract	Section 0 and Appendix A

Base URL and transport

Property	Value
Protocol	HTTP (no TLS in the reference deployment)
Default port	5030
Base URL	http://<host>:5030
Authentication	None — see Appendix D
Content type	application/json
Encoding	UTF-8

Note The 1.ID specification writes paths as `api/checkin/`. This build serves them without the `api/` prefix — the correct path is `/checkin`. `GET /routes` returns the authoritative list.

Request headers

Key	Value
Content-Type	application/json

The message envelope

Every business request carries these five fields alongside its own payload:

```
{
  "MessageType" : "CheckinRequest",
  "ClientUID"   : "Establishment101",
  "MessageUID"  : null,
  "CorrelationUID" : "f50a8b4d",
  "Timestamp"   : "2026-08-05T08:00:00"
}
```

Field	Required	Notes
MessageType	Yes	Free text. Echoed back into the response; never validated.
ClientUID	Yes	The hotel's DTCM registration id. Not validated by this build — any string is accepted.
MessageUID	Yes	Send null. The API generates one and returns it.
CorrelationUID	Yes	Your identifier. Returned unchanged — use it to pair request and response, and quote it in any support request.
Timestamp	Yes	ISO 8601. Not validated.

A successful business response carries the same envelope, lower-cased, plus the endpoint's own fields:

```
{
  "hasErrors"      : false,
  "errorMessages"  : {},
  "messageType"    : "CheckinResponse",
  "clientUID"      : "Establishment101",
  "messageUID"     : "7230ad75-8a9f-45c0-a000-ddb587c66345",
  "correlationUID" : "f50a8b4d",
  "timestamp"      : "2026-08-05T11:40:01.125047+05:30"
}
```

Important Three endpoints break this pattern. POST /checkout, POST /roomchange and POST /checkincancellation return a bare {"CheckinUID": n} on success. Only their failures use the envelope.

How failures are reported

Validation failures return HTTP 400 with hasErrors true and a numbered code in errorMessages.general:

```
{
  "hasErrors"      : true,
  "errorMessages"  : { "general": "40002 Checkin room does not exist" },
  "messageType"    : "CheckinResponse",
  "clientUID"      : "Establishment101",
  "messageUID"     : "9c1f0f3e-...",
  "correlationUID" : "f50a8b4d",
  "timestamp"      : "2026-08-05T11:40:01.125047+05:30"
}
```

Important Not every failure carries a number. All eight overlap rules — a check-in that clashes with another stay in the same room, a room change outside the stay window, and so on — return a free-text sentence naming the conflicting record and its times. Your error handling must accept both forms. See Appendix A.

Identifiers: the one trap that costs the most time

The field named uid **does not mean the same thing in every response**.

Response	guests[].uid is...
POST /checkin	the checkingest id — a row in the guest-to-stay join table
POST /guest, GET /guest	the guest id
Required by POST /guestattachment, POST /guestcheckout, NewMainGuestUID	the guest id

Always take guest identifiers from **GET /guest**. Using the value returned by POST /checkin will address the wrong record, and the API will not complain.

Reference tables

The 1.ID specification distributes these tables as a shared spreadsheet. This build serves them as live endpoints, which is preferable — read them at start-up rather than hard-coding the values below.

Countries are matched on their ISO two-letter code (TwoCode). Every other table is matched on DtcnCode.

Code sets

Field	Endpoint	Valid values
PaymentMethodCode	/paymenttype	cash, creditcard
CreditCardTypeCode	/cardtype	visa (13–16 digits), mastercard (16), amex (15), diners (14–16)
GenderCode	—	male, female
RelationshipCode	/relationship	businessAssociate, familyHelper, familyMember, friend, visitor
VisitPurposeCode	/visitpurpose	tourism, visitor, business, medical, religious, Athlete, crewmember, hotelstaff, medicalstaff, otherstaff, patients
AttachmentTypeCode	/documenttype	21 values: passport, emiratesid, uaedrivinglicence, identificationcardgcc, diplomaticpassport, privatepassport, temporarypassport, servicepassport, politicalpassport, seamanpassport, unpassport, traveldocument, lebanesetraveldocument, iraqitraveldocument, syriantraveldocument, egyptiantraveldocument, generaldeclarationform, labourcarduae, diplomaticcard, airlineid, militarycard
EmirateCode	/emirate	dubai, abudhabi, sharjah, fujirah, rak, ajman, ummalquwain, alain, westernareas
AccessibilityTypes[.Code	/accessibilitytype	hearing, visual, mobility, other
CancellationReasonCode	/cancellationreason	Unhappy (Unsatisfied Guest), UserError (User Error)
NationalityCode, ResidenceCountryCode, IssueCountryCode	/country	ISO two-letter, 252 rows — AE, GB, US, IN, LK, ...

Warning Athlete is capitalised in the table — send it exactly as shown. Visa Electron exists in /cardtype but its DtcnCode is the string "0", which is a data defect; do not use it.

Escort types

GET /escorttype returns 1 ChildEscort and 2 AdultEscort. **No payload field selects between them.** This build writes EscortTypeid 2 in every case, so ChildEscort is never recorded and the AdultEscortCount / ChildEscortCount fields on a check-in are always null. See Appendix B, item 10.

Common structures

Two composite types recur throughout this specification.

GuestInfo

Used as each element of the `Guests[]` array on the check-in endpoints, and as the `GuestInfo` object on the guest endpoints. The shape is identical in both places.

Property	Type	Required	Rule
UID	string	Yes	Send null on create; ignored.
GuestCode	string	Yes	Your identifier for this guest, ≤ 75 characters. Duplicates are accepted by this build.
FirstName	string	Yes	English letters and spaces only. No digits, no double spaces.
LastName	string	Yes	Same rule. ≤ 45 characters on PUT /checkin.
ArabicName	string	Yes	Arabic characters and spaces only. Required on the check-in endpoints; an empty string fails. Not stored — the API transliterates the English names instead.
GenderCode	string	Yes	male or female. Stored capitalised.
Email	string	Key required	May be null. If present, must contain "@".
Mobile	string	Yes	Digits only. No "+", no spaces.
ResidenceCountryPhone	string	Key required	Free text.
BirthDate	DateTime	Yes	ISO 8601. Must not be in the future.
BirthPlace	string	Yes	Letters only, no spaces. "Abu Dhabi" is rejected; "AbuDhabi" is accepted.
NationalityCode	string	Yes	ISO two-letter, must exist in the country table.
ResidenceCountryCode	string	Yes	ISO two-letter. Also determines the stored MobileCode.
RelationshipCode	string	Conditional	Required for every non-main guest. Send null for the main guest — the check is skipped when <code>IsMainGuest</code> is true.
VisitPurposeCode	string	Yes	Must exist in the visit purpose table.
CheckinDateTime	DateTime	Yes	At or after the transaction's <code>CheckinDateTime</code> ; never in the future.
CheckoutDateTime	DateTime	Key required	null everywhere except <code>AddBackdatedCheckin</code> .
IsMainGuest	bool	Yes	Exactly one true per check-in transaction.
AttachmentTypeCode	string	Yes	Must exist in the document type table.
DocumentNumber	string	Yes	Alphanumeric, no spaces. This is the guest's identity — see the note below.
IssueCountryCode	string	Yes	ISO two-letter.
IssueDate	DateTime	Yes	Not in the future; strictly before <code>ExpiryDate</code> .
ExpiryDate	DateTime	Yes	Must not be in the past.

Property	Type	Required	Rule
EmirateCode	string	Conditional	Required when IssueCountryCode is AE — whatever the document type. See Appendix A.
RequiresAccessibility	bool	No	true stores the accessibility detail; false or absent stores null.
AccessibilityTypes	array	No	[{"Code": "mobility"}, ...]
OtherAccessibilityType	string	Conditional	Required when AccessibilityTypes contains "other".
Attachments	array	Yes	At least one AttachmentInfo. An empty array or null is rejected.

Warning Seven fields are read without a default — GenderCode, Email, ResidenceCountryPhone, DocumentNumber, ResidenceCountryCode, CheckinDateTime and IsMainGuest. Omitting any of them produces an HTTP 500, not a tidy 400. Always send all seven, even when the value is null.

Important DocumentNumber is the guest's identity. Posting a guest whose document number already exists updates that guest rather than creating a new one. Reusing it on the same stay returns 50016.

AttachmentInfo

```
{
  "Id"           : 0,
  "UID"          : null,
  "AttachmentCode" : "PC-DOC-0001",
  "Name"         : "Image_1.jpg",
  "Size"         : 160,
  "ContentBase64Encoded" : "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcGBQgHBwcJCQgKDBQNDAsLDBkSEw8UHR..."
}
```

Property	Type	Required	Rule
Id	int	Yes	Send 0 on create.
UID	string	Yes	Send null on create.
AttachmentCode	string	Yes	Your identifier. The build also accepts the spelling attachmenttCode used in the 1.ID specification's samples.
Name	string	Yes	Becomes the filename on disk. See the note below.
Size	int	Yes	Declared size in bytes. Not cross-checked against the content.
ContentBase64Encoded	string	Yes	Base64 JPEG. Must be under 200 KB decoded, or 60004.

Images are resized to 1080 × 720 and re-encoded at quality 85. They are written under BaseDirectoryPath on the API host and stored as a blob in the database.

Warning POST /checkin and POST /guest do not rename colliding filenames, so a guest's second document must use a distinct Name. POST /guestattachment does de-duplicate — a second Image_2.jpg becomes Image_2_1.jpg.

1. Checkin

The check-in transaction is the entry point for everything else: it is the only operation that creates a CheckinUID, and every other endpoint in this specification requires one.

1.1 Checkin GET

1.1.1 Request body sample

```
{
  "CheckinUID": "6",
  "MessageType": "CheckinStatusRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkin-get-001",
  "Timestamp": "2026-08-04T08:00:00"
}
```

Important This GET carries a JSON body. Postman and curl.exe send it correctly; PowerShell 5.1 refuses outright with "Cannot send a content-body with this verb-type".

1.1.2 Request

Method	URL
GET	/checkin

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The unique identifier of the check-in record.

1.1.3 Response

200 — an object describing the check-in:

```
{
  "hasErrors"      : false,
  "messageType"    : "CheckinRequest",
  "clientUID"      : "Establishment101",
  "correlationUID" : "test-checkin-get-001",
  "data": {
    "Id"           : 6,
    "CheckinUID"   : null,
    "RoomNumber"   : "001",
    "CheckinDate"  : "2026-05-18T13:52:55",
    "IsActive"     : true,
    "ChargeExtra"  : false,
    "IsFeeUpdated" : false,
    "TDFee"        : null,
    "AddedAt"      : "2026-05-18T13:52:55",
    "AddedFrom"    : "FRONTDESK-PC",
    "CheckinTypeId": 1,
    "PaymentId"    : 6,
    "ChildEscortCount" : null,
    "AdultEscortCount" : null
  }
}
```

Code	Meaning / condition
404	Check-in not found — the CheckinUID does not exist.

1.2 Checkin POST

Creates the payment record, the check-in, each guest, their documents, the guest-to-stay rows, a log entry and a version snapshot per guest, then flags the room as occupied.

1.2.1 Request body sample — one guest, cash

```
{
  "RoomNumber": "003",
  "IsEarlyCheckin": false,
  "CheckinDateTime": "2026-08-04T08:00:00",
  "PaymentMethodCode": "cash",
  "CreditCardNumber": null,
  "CreditCardTypeCode": null,
  "PaidAmount": 0,
  "Comment": "payload collection - simple cash checkin",
  "IsWaitingForRoom": false,
  "IsHouseUse": false,
  "MessageType": "CheckinRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkin-cash-001",
  "Timestamp": "2026-08-04T08:00:00",
  "Guests": [
    {
      "UID": null,
      "GuestCode": "PC-MAIN-0001",
      "FirstName": "John",
      "LastName": "Smith",
      "ArabicName": "جون سميث",
      "GenderCode": "male",
      "Email": "john.smith@example.com",
      "Mobile": "501234567",
      "ResidenceCountryPhone": "971501234567",
      "BirthDate": "1985-06-14T00:00:00",
      "BirthPlace": "London",
      "NationalityCode": "GB",
      "ResidenceCountryCode": "AE",
      "RelationshipCode": null,
      "VisitPurposeCode": "tourism",
      "CheckinDateTime": "2026-08-04T08:00:00",
      "CheckoutDateTime": null,
      "IsMainGuest": true,
      "AttachmentTypeCode": "passport",
      "DocumentNumber": "GB9876543",
      "IssueCountryCode": "GB",
      "IssueDate": "2020-01-15T00:00:00",
      "ExpiryDate": "2030-01-15T00:00:00",
      "EmirateCode": null,
      "RequiresAccessibility": false,
      "AccessibilityTypes": [],
      "OtherAccessibilityType": "",
      "Attachments": [
        {
          "Id": 0,
          "UID": null,
          "AttachmentCode": "PC-DOC-MAIN-0001",
          "Name": "Image_1.jpg",
          "Size": 160,
          "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
        }
      ]
    }
  ]
}
```

1.2.2 Request

Method	URL
POST	/checkin

Parameter	Type	Required	Notes
RoomNumber	string	Yes	Must exist and be free.
IsEarlyCheckin	bool	Yes	Applies an extra tourism dirham fee for one additional night.
CheckinDateTime	DateTime	Yes	Arrival date and time. Must not be in the future.

Parameter	Type	Required	Notes
Guests	IList<GuestInfo>	Yes	At least one; exactly one with IsMainGuest true.
PaymentMethodCode	string	Yes	cash or creditcard. Must be null when IsHouseUse is true.
CreditCardNumber	string	Yes	Required when PaymentMethodCode is creditcard, otherwise null.
CreditCardTypeCode	string	Yes	Required when PaymentMethodCode is creditcard, otherwise null.
PaidAmount	number	No	Not present in the 1.ID specification. Defaults to 0.
IsWaitingForRoom	bool	Yes	Always send false.
IsHouseUse	bool	Yes	See the warning below.
Comment	string	No	Free-text remarks.

Warning IsHouseUse must be present. The handler reads it with a default of true, so omitting the key makes the API treat an ordinary paid check-in as house use and reject it with 40035.

1.2.3 Validation order

Checks run in this order and the first failure returns immediately — later checks never execute. Knowing the order tells you which error you will see when a payload is wrong in more than one way.

#	Code	Check
1	40002	Room exists
2	40003	Room is active
3	40050	Room is not already occupied
4	—	Check-in does not overlap the room's previous stay (free text)
5	40004	CheckinDateTime is not in the future
6	40005	At least one guest supplied
7	40006	A main guest is present
8	40028	Exactly one main guest
9	50014	No guest checks in before the transaction
10	40007	Payment method is cash or creditcard
11	40008	Card number present when paying by card
12–15	40035–40038	House use rejects payment method, card number, early check-in, waiting for room
16	40033	At least one guest checks in at exactly the transaction time
17	50010	Guest object present
18–30	50001–50015	Per-guest field checks, in the order listed in GuestInfo
31–41	60001–60013	Per-attachment checks: presence, document number, type, dates, country, emirate, size, accessibility text

1.2.4 Response

200 — the envelope plus the created identifiers:

```
{
  "CheckinUID"      : 17,
  "clientUID"       : "Establishment101",
  "correlationUID"  : "test-checkin-cash-001",
  "errorMessages"   : {},
  "guests": [
    {
      "guestCode"    : "PC-MAIN-0001",
      "uid"          : 13,
      "attachments"  : [ { "attachmentCode": "PC-DOC-MAIN-0001", "uid": 5 } ]
    }
  ],
  "hasErrors"       : false,
  "messageType"     : "CheckinResponse",
  "messageUID"      : "7230ad75-8a9f-45c0-a000-ddb587c66345",
  "requestMessageID" : 29,
  "timestamp"       : "2026-08-05T11:40:01.125047+05:30"
}
```

Important Capture CheckinUID — every downstream call needs it. The uid inside guests[] is the checkinguest id, not the guest id; call GET /guest to obtain guest ids.

Code	Meaning / condition
40002	Checkin room does not exist
40003	Checkin room is not available
40004	Checkin date must not be in future
40005	Checkin no guests specified
40006	Checkin no main guest specified
40007	Checkin invalid payment method
40008	Checkin invalid credit card number
40028	You must have only one main guest
40033	Check-in transaction must have at least one guest with check-in date and time equal to the transaction's
40035	Check-in with house use flag does not require payment method
40036	Check-in with house use flag does not require credit card number
40037	Check-in with house use flag does not require early checkin
40038	Check-in with house use flag not allowed with waiting for room
40050	Checkin room is occupied — not defined in the 1.ID specification
50001	Guest invalid guest code, or invalid first name
50002	Guest invalid last name
50005	Guest invalid nationality
50006	Guest invalid residence country
50007	Guest invalid birth date
50008	Guest invalid visit purpose
50009	Guest invalid relationship
50010	Guest is not specified
50011	Guest mobile number is invalid
50013	Birth place is not specified

Code	Meaning / condition
50014	Guest check-in date is before the transaction check-in date
50015	Guest check-in date time should not be in future
60001	Attachment is not specified
60004	Attachment should not exceed 200 KB in size
60006	Attachment document number is invalid
60007	Attachment type is not specified
60008	Attachment issue date in the future, or missing "other" accessibility description
60009	Attachment issue date should be before expiry date
60010	Attachment issue country is not specified
60011	Attachment expiry date is not specified, or the document is expired
60012	Attachment issue emirate is not specified
60013	Attachment issue date is not specified

1.2.5 Variant — main guest with two escorts

Additional guests are simply further entries in `Guests[]` with `IsMainGuest` false and a `RelationshipCode`. This is the first of the two ways to add an escort; the second is Section 4.2.

```
{
  "RoomNumber": "005",
  "IsEarlyCheckin": false,
  "CheckinDateTime": "2026-08-04T08:00:00",
  "PaymentMethodCode": "cash",
  "CreditCardNumber": null,
  "CreditCardTypeCode": null,
  "PaidAmount": 0,
  "Comment": "payload collection - main guest plus two escorts in one transaction",
  "IsWaitingForRoom": false,
  "IsHouseUse": false,
  "MessageType": "CheckinRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkin-escorts-001",
  "Timestamp": "2026-08-04T08:00:00",
  "Guests": [
    {
      "UID": null,
      "GuestCode": "PC-MAIN-0100",
      "FirstName": "Michael",
      "LastName": "Brown",
      "ArabicName": "مايكل براون",
      "GenderCode": "male",
      "Email": "michael.brown@example.com",
      "Mobile": "501110001",
      "ResidenceCountryPhone": "971501110001",
      "BirthDate": "1980-02-11T00:00:00",
      "BirthPlace": "Manchester",
      "NationalityCode": "GB",
      "ResidenceCountryCode": "AE",
      "RelationshipCode": null,
      "VisitPurposeCode": "business",
      "CheckinDateTime": "2026-08-04T08:00:00",
      "CheckoutDateTime": null,
      "IsMainGuest": true,
      "AttachmentTypeCode": "passport",
      "DocumentNumber": "GB1000001",
      "IssueCountryCode": "GB",
      "IssueDate": "2019-03-01T00:00:00",
      "ExpiryDate": "2029-03-01T00:00:00",
      "EmirateCode": null,
      "RequiresAccessibility": false,
      "AccessibilityTypes": [],
      "OtherAccessibilityType": "",
      "Attachments": [
        {
          "Id": 0,
          "UID": null,
          "AttachmentCode": "PC-D0C-0100",

```

```

        "Name": "Image_1.jpg",
        "Size": 160,
        "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
    }
}
},
{
    "UID": null,
    "GuestCode": "PC-ESCORT-0101",
    "FirstName": "Sarah",
    "LastName": "Brown",
    "ArabicName": "سارة براون",
    "GenderCode": "female",
    "Email": "sarah.brown@example.com",
    "Mobile": "501110002",
    "ResidenceCountryPhone": "971501110002",
    "BirthDate": "1983-07-22T00:00:00",
    "BirthPlace": "Manchester",
    "NationalityCode": "GB",
    "ResidenceCountryCode": "AE",
    "RelationshipCode": "familyMember",
    "VisitPurposeCode": "tourism",
    "CheckinDateTime": "2026-08-04T08:05:00",
    "CheckoutDateTime": null,
    "IsMainGuest": false,
    "AttachmentTypeCode": "passport",
    "DocumentNumber": "GB1000002",
    "IssueCountryCode": "GB",
    "IssueDate": "2019-03-01T00:00:00",
    "ExpiryDate": "2029-03-01T00:00:00",
    "EmirateCode": null,
    "RequiresAccessibility": true,
    "AccessibilityTypes": [
        { "Code": "mobility" }
    ],
    "OtherAccessibilityType": "",
    "Attachments": [
        {
            "Id": 0,
            "UID": null,
            "AttachmentCode": "PC-D0C-0101",
            "Name": "Image_1.jpg",
            "Size": 160,
            "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
        }
    ]
}
},
{
    "UID": null,
    "GuestCode": "PC-ESCORT-0102",
    "FirstName": "Omar",
    "LastName": "Al Mansoori",
    "ArabicName": "عمر المنصوري",
    "GenderCode": "male",
    "Email": "omar.mansoori@example.com",
    "Mobile": "501110003",
    "ResidenceCountryPhone": "971501110003",
    "BirthDate": "1990-11-02T00:00:00",
    "BirthPlace": "Dubai",
    "NationalityCode": "AE",
    "ResidenceCountryCode": "AE",
    "RelationshipCode": "businessAssociate",
    "VisitPurposeCode": "business",
    "CheckinDateTime": "2026-08-04T08:10:00",
    "CheckoutDateTime": null,
    "IsMainGuest": false,
    "AttachmentTypeCode": "emiratesid",
    "DocumentNumber": "784199012345678",
    "IssueCountryCode": "AE",
    "IssueDate": "2022-05-10T00:00:00",
    "ExpiryDate": "2032-05-10T00:00:00",
    "EmirateCode": "dubai",
    "RequiresAccessibility": false,
    "AccessibilityTypes": [],
    "OtherAccessibilityType": "",
    "Attachments": [
        {
            "Id": 0,
            "UID": null,
            "AttachmentCode": "PC-D0C-0102",
            "Name": "Image_1.jpg",
            "Size": 160,
            "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
        }
    ]
}
}
}
}

```

1.2.6 Variant — credit card, early check-in, accessibility

```
{
  "RoomNumber": "007",
  "IsEarlyCheckin": true,
  "CheckinDateTime": "2026-08-04T08:00:00",
  "PaymentMethodCode": "creditcard",
  "CreditCardNumber": "4111111111111111",
  "CreditCardTypeCode": "visa",
  "PaidAmount": 250.5,
  "Comment": "payload collection - credit card + early checkin",
  "IsWaitingForRoom": false,
  "IsHouseUse": false,
  "MessageType": "CheckinRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkin-card-001",
  "Timestamp": "2026-08-04T08:00:00",
  "Guests": [
    {
      "UID": null,
      "GuestCode": "PC-MAIN-0200",
      "FirstName": "Priya",
      "LastName": "Nair",
      "ArabicName": "بريا ناير",
      "GenderCode": "female",
      "Email": "priya.nair@example.com",
      "Mobile": "501220001",
      "ResidenceCountryPhone": "971501220001",
      "BirthDate": "1992-09-30T00:00:00",
      "BirthPlace": "Kochi",
      "NationalityCode": "IN",
      "ResidenceCountryCode": "IN",
      "RelationshipCode": null,
      "VisitPurposeCode": "medical",
      "CheckinDateTime": "2026-08-04T08:00:00",
      "CheckoutDateTime": null,
      "IsMainGuest": true,
      "AttachmentTypeCode": "passport",
      "DocumentNumber": "IN7788990",
      "IssueCountryCode": "IN",
      "IssueDate": "2021-06-01T00:00:00",
      "ExpiryDate": "2031-06-01T00:00:00",
      "EmirateCode": null,
      "RequiresAccessibility": true,
      "AccessibilityTypes": [
        { "Code": "other" }
      ],
      "OtherAccessibilityType": "Requires a wheelchair ramp",
      "Attachments": [
        {
          "Id": 0,
          "UID": null,
          "AttachmentCode": "PC-D0C-0200",
          "Name": "Image_1.jpg",
          "Size": 160,
          "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
        }
      ]
    }
  ]
}
```

1.2.7 Variant — house use

Every payment field is null and IsHouseUse is true. A house-use stay also refuses a late check-out later, with 40039.

```
{
  "RoomNumber": "008",
  "IsEarlyCheckin": false,
  "CheckinDateTime": "2026-08-04T08:00:00",
  "PaymentMethodCode": null,
  "CreditCardNumber": null,
  "CreditCardTypeCode": null,
  "PaidAmount": 0,
  "Comment": "payload collection - house use stay, no payment",
  "IsWaitingForRoom": false,
  "IsHouseUse": true,
  "MessageType": "CheckinRequest",
  "ClientUID": "Establishment101",
}
```

```

"MessageUID": null,
"CorrelationUID": "test-checkin-houseuse-001",
"Timestamp": "2026-08-04T08:00:00",
"Guests": [
  {
    "UID": null,
    "GuestCode": "PC-MAIN-0300",
    "FirstName": "Fatima",
    "LastName": "Hassan",
    "ArabicName": "فاطمة حسن",
    "GenderCode": "female",
    "Email": "fatima.hassan@example.com",
    "Mobile": "501330001",
    "ResidenceCountryPhone": "971501330001",
    "BirthDate": "1995-01-20T00:00:00",
    "BirthPlace": "Sharjah",
    "NationalityCode": "AE",
    "ResidenceCountryCode": "AE",
    "RelationshipCode": null,
    "VisitPurposeCode": "hotelstaff",
    "CheckinDateTime": "2026-08-04T08:00:00",
    "CheckoutDateTime": null,
    "IsMainGuest": true,
    "AttachmentTypeCode": "emiratesid",
    "DocumentNumber": "784199512345671",
    "IssueCountryCode": "AE",
    "IssueDate": "2023-01-01T00:00:00",
    "ExpiryDate": "2033-01-01T00:00:00",
    "EmirateCode": "sharjah",
    "RequiresAccessibility": false,
    "AccessibilityTypes": [],
    "OtherAccessibilityType": "",
    "Attachments": [
      {
        "Id": 0,
        "UID": null,
        "AttachmentCode": "PC-D0C-0300",
        "Name": "Image_1.jpg",
        "Size": 160,
        "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
      }
    ]
  }
]
}

```

1.3 Checkin PUT

Amends a stay that is still open.

1.3.1 Request body sample

```

{
  "UID": "REPLACE_WITH_CHECKIN_UID",
  "RoomNumber": "003",
  "IsEarlyCheckin": false,
  "CheckinDateTime": "2026-08-04T08:00:00",
  "IsLateCheckout": null,
  "CheckoutDateTime": null,
  "PaymentMethodCode": "cash",
  "CreditCardNumber": null,
  "CreditCardTypeCode": null,
  "PaidAmount": 0,
  "Comment": "payload collection - amend the main guest of an open checkin",
  "IsWaitingForRoom": null,
  "IsHouseUse": false,
  "MessageType": "CheckinUpdateRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkin-put-001",
  "Timestamp": "2026-08-04T08:15:00",
  "Guests": [
    {
      "UID": null,
      "GuestCode": "PC-MAIN-0001",
      "FirstName": "Jonathan",
      "LastName": "Smith",
      "ArabicName": "جون سميث",
      "GenderCode": "male",
      "Email": "jonathan.smith@example.com",
      "Mobile": "501234599",
      "ResidenceCountryPhone": "971501234599",
      "BirthDate": "1985-06-14T00:00:00",
      "BirthPlace": "London",
      "NationalityCode": "GB",
    }
  ]
}

```



```

    "ResidenceCountryCode": "AE",
    "RelationshipCode": null,
    "VisitPurposeCode": "business",
    "CheckinDateTime": "2026-08-04T08:00:00",
    "CheckoutDateTime": null,
    "IsMainGuest": true,
    "AttachmentTypeCode": "passport",
    "DocumentNumber": "GB9876543",
    "IssueCountryCode": "GB",
    "IssueDate": "2020-01-15T00:00:00",
    "ExpiryDate": "2030-01-15T00:00:00",
    "EmirateCode": null,
    "RequiresAccessibility": false,
    "AccessibilityTypes": [],
    "OtherAccessibilityType": "",
    "Attachments": [
      {
        "Id": 0,
        "UID": null,
        "AttachmentCode": "PC-DOC-MAIN-0001",
        "Name": "Image_1.jpg",
        "Size": 160,
        "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG... "
      }
    ]
  }
}
}
}

```

1.3.2 Request

Method	URL
PUT	/checkin

Parameter	Type	Required	Notes
UID	string	Yes	The check-in identifier. This endpoint uses UID, not CheckinUID.
Guests	IList<GuestInfo>	Yes	Full guest objects, not a delta. Exactly one main guest.
CheckoutDateTime	DateTime	Yes	Must be null — supplying a value returns 40023.
IsLateCheckout	bool	No	Send null.
RoomNumber	string	No	May be null to leave the room unchanged.
CheckinDateTime	DateTime	Yes	Cannot move earlier than the stay's current check-in time.

Warning This endpoint identifies the check-in by UID. Every other endpoint in this specification uses CheckinUID.

1.3.3 Response

200 — the same shape as 1.2.4, with messageType "CheckinUpdateResponse".

Code	Meaning / condition
40002 / 40003	Room does not exist / is not available
40004	Checkin date must not be in future
40006 / 40028	No main guest / more than one main guest
40008	Invalid credit card number

Code	Meaning / condition
40011	Checkin is already checked out
40023	You cannot set a CheckoutDateTime during a check-in update
50001–50015	Per-guest checks, as 1.2
60001–60013	Per-attachment checks, as 1.2

Warning Known defect. This handler locates the row to update using the guest's most recent record across every check-in, not the record belonging to the check-in being amended. A repeat visitor who is also on a newer stay will have the wrong stay modified. Keep test guests unique per stay. See Appendix B, item 3.

1.4 Checkin AddBackdatedCheckin POST

Note This endpoint is an extension of this build. It does not appear in the 1.ID API Specification Document.

Records a stay that has already started and ended — used when reconciling paper records. The usual room-overlap rules are relaxed, but the requested window must still not clash with the room's existing history.

1.4.1 Request body sample

```
{
  "RoomNumber": "020",
  "IsEarlyCheckin": false,
  "IsLateCheckout": false,
  "CheckinDateTime": "2026-08-01T10:00:00",
  "CheckoutDateTime": "2026-08-02T11:00:00",
  "PaymentMethodCode": "cash",
  "CreditCardNumber": null,
  "CreditCardTypeCode": null,
  "PaidAmount": 0,
  "Comment": "payload collection - stay that already started and ended",
  "IsWaitingForRoom": false,
  "IsHouseUse": false,
  "MessageType": "AddBackdatedCheckinRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-backdated-001",
  "Timestamp": "2026-08-04T08:00:00",
  "Guests": [
    {
      "UID": null,
      "GuestCode": "PC-BACK-0400",
      "FirstName": "Daniel",
      "LastName": "Perera",
      "ArabicName": "دانيال بيريرا",
      "GenderCode": "male",
      "Email": "daniel.perera@example.com",
      "Mobile": "501440001",
      "ResidenceCountryPhone": "94771440001",
      "BirthDate": "1988-04-18T00:00:00",
      "BirthPlace": "Colombo",
      "NationalityCode": "LK",
      "ResidenceCountryCode": "LK",
      "RelationshipCode": null,
      "VisitPurposeCode": "tourism",
      "CheckinDateTime": "2026-08-01T10:00:00",
      "CheckoutDateTime": "2026-08-02T11:00:00",
      "IsMainGuest": true,
      "AttachmentTypeCode": "passport",
      "DocumentNumber": "LK5566778",
      "IssueCountryCode": "LK",
      "IssueDate": "2020-02-01T00:00:00",
      "ExpiryDate": "2030-02-01T00:00:00",
      "EmirateCode": null,
      "RequiresAccessibility": false,
      "AccessibilityTypes": [],
      "OtherAccessibilityType": "",
      "Attachments": [
```

```

    {
      "Id": 0,
      "UID": null,
      "AttachmentCode": "PC-D0C-0400",
      "Name": "Image_1.jpg",
      "Size": 160,
      "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG... "
    }
  ]
}

```

1.4.2 Request

Method	URL
POST	/checkin/AddBackdatedCheckin

Parameter	Type	Required	Notes
CheckoutDateTime	DateTime	Yes	Required at the transaction level and on every guest.
(all others)	—	—	As POST /checkin.

1.4.3 Response

200 — as 1.2.4. In addition to the codes listed there:

Code	Meaning / condition
40014	CheckoutDateTime is required
40034	Check-out must be after check-in
40004	Neither date may be in the future
40035	Each guest's window must sit inside the transaction's window — note this code is reused, see Appendix A
50014	Guest check-in or check-out falls outside the stay

2. Checkout

2.1 Checkout POST

Closes the stay, frees the room, stamps a check-out date on every guest still present, and writes a check-out record with CheckoutTypeId 1.

2.1.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "IsLateCheckout": false,
  "CheckoutDateTime": "2026-08-04T10:30:00",
  "Comment": "payload collection - close the stay",
  "MessageType": "CheckoutRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkout-001",
  "Timestamp": "2026-08-04T10:30:00"
}
```

2.1.2 Request

Method	URL
POST	/checkout

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The check-in to close.
IsLateCheckout	bool	Yes	Applies an extra tourism dirham fee. Rejected on a house-use stay.
CheckoutDateTime	DateTime	Yes	Departure date and time.
Comment	string	No	Free-text remarks.

2.1.3 Ordering rules

CheckoutDateTime must be at or after every one of the following, or the call is refused:

Must be after	Reported as
Every guest's check-in date and time	40034
Every guest check-out already recorded	40034
The most recently added escort's check-in and check-out	free text
Any room change on this stay	free text
Any main-guest change on this stay	free text

2.1.4 Response

200 — a bare object, not the envelope:

```
{ "CheckinUID": 17 }
```

Code	Meaning / condition
40004	Check-out date must not be in future

Code	Meaning / condition
40009	Checkin is not specified
40010	Checkin is already cancelled
40011	Checkin is already checked out
40014	Checkout date time is invalid
40034	Check-out must be at or after any guest check-in or guest check-out
40039	Check-out with house use flag does not require late checkout

2.2 Checkout PUT

Adjusts a check-out that has already happened. The mirror image of 2.1: the stay must be inactive.

2.2.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "IsLateCheckout": true,
  "CheckoutDateTime": "2026-08-04T11:00:00",
  "MessageType": "CheckoutUpdateRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-checkout-put-001",
  "Timestamp": "2026-08-04T11:00:00"
}
```

2.2.2 Request

Method	URL
PUT	/checkout

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The closed check-in to amend.
IsLateCheckout	bool	Yes	
CheckoutDateTime	DateTime	Yes	The corrected departure date and time.

2.2.3 Response

```
{ "CheckinUID": 17 }
```

Code	Meaning / condition
40004	Check-out date must not be in future
40009	Checkin is not specified
40010	Checkin is already cancelled
40014	Checkout date time is invalid
40031	Check-in is still active — use POST /checkout instead
40032	Check-out must be at or after any guest check-out
40039	House use does not require late checkout

3. Checkin Cancellation

3.1 CheckinCancellation POST

Cancels a check-in rather than closing it: writes a check-out record with CheckoutTypeid 2 and a cancellation reason, frees the room, and closes out every guest.

3.1.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "CancellationDateTime": "2026-08-04T09:00:00",
  "CancellationReasonCode": "UserError",
  "Comment": "payload collection - cancel a checkin created for testing",
  "MessageType": "CheckinCancellationRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-cancel-001",
  "Timestamp": "2026-08-04T09:00:00"
}
```

3.1.2 Request

Method	URL
POST	/checkincancellation

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The check-in to cancel.
CancellationDateTime	DateTime	Yes	When the cancellation takes effect.
CancellationReasonCode	string	Yes	Unhappy or UserError.
Comment	string	No	Free-text remarks.

3.1.3 Response

```
{ "CheckinUID": 17 }
```

Code	Meaning / condition
40004	Cancellation date must not be in future
40009	Checkin is not specified
40010	Checkin is already cancelled
40011	Checkin is already checked out
40025	Cancellation date time is not specified, or precedes the check-in
—	Free text: the cancellation must fall within the same calendar month as the check-in

Note The same-month restriction is an addition of this build and is not in the 1.ID specification. Code for it.

4. Guest

This section covers reading the guests on a stay, and adding or amending an individual guest after check-in. Adding a guest here is the second of the two escort paths — the first is Section 1.2.5.

4.1 Guest GET

4.1.1 Request body sample

```
{
  "CheckinUID": "6",
  "MessageType": "GuestStatusRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-guest-get-001",
  "Timestamp": "2026-08-04T08:00:00"
}
```

Important This GET also carries a JSON body. It is the only reliable source of guest ids — the values needed by Sections 5 and 6, and by NewMainGuestUID.

4.1.2 Request

Method	URL
GET	/guest

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The check-in whose guests you want.

4.1.3 Response

200 — every guest on the stay, each with their document metadata, isMainGuest flag and guest id.

Code	Meaning / condition
404	Check-in not found

4.2 Guest POST — add a guest or escort

Adds one guest to a stay that is already open. This is the "add escort" action in the desktop client.

4.2.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "Comment": "payload collection - add an escort to an open checkin",
  "MessageType": "GuestRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-add-escort-001",
  "Timestamp": "2026-08-04T08:30:00",
  "GuestInfo": {
    "UID": null,
    "GuestCode": "PC-ESCORT-0500",
    "FirstName": "Alan",
    "LastName": "Turing",
    "ArabicName": "آلان تورينج",
    "GenderCode": "male",
    "Email": "alan.turing@example.com",
    "Mobile": "501550001",
    "ResidenceCountryPhone": "971501550001",
  }
}
```

```

"BirthDate": "1990-11-20T00:00:00",
"BirthPlace": "Manchester",
"NationalityCode": "GB",
"ResidenceCountryCode": "AE",
"RelationshipCode": "friend",
"VisitPurposeCode": "visitor",
"CheckinDateTime": "2026-08-04T08:30:00",
"CheckoutDateTime": null,
"IsMainGuest": false,
"AttachmentTypeCode": "passport",
"DocumentNumber": "GB1122334",
"IssueCountryCode": "GB",
"IssueDate": "2019-05-01T00:00:00",
"ExpiryDate": "2029-05-01T00:00:00",
"EmirateCode": null,
"RequiresAccessibility": false,
"AccessibilityTypes": [],
"OtherAccessibilityType": "",
"Attachments": [
  {
    "Id": 0,
    "UID": null,
    "AttachmentCode": "PC-DOC-0500",
    "Name": "Image_1.jpg",
    "Size": 160,
    "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
  }
]
}

```

4.2.2 Request

Method	URL
POST	/guest

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The open check-in to add to.
GuestInfo	GuestInfo	Yes	A single object, not an array. One guest per request.
Comment	string	No	Free-text remarks.

Warning The 1.ID specification's sample for this endpoint sends "Attachments": null and "RelationshipCode": "Son". Both are rejected by this build — at least one attachment is required (60001), and Son is not a value in the relationship table (50009).

4.2.3 Response

200 — the envelope with the created guest. Note that uid here is the guest id, the opposite of Section 1.2.4.

```

{
  "CheckinUID"      : 17,
  "clientUID"       : "Establishment101",
  "correlationUID"  : "test-add-escort-001",
  "errorMessages"   : {},
  "guests": [
    {
      "guestCode"    : "PC-ESCORT-0500",
      "uid"          : 14,
      "attachments"  : [ { "attachmenttCode": "PC-DOC-0500", "uid": 9 } ]
    }
  ],
  "hasErrors"       : false,
  "messageType"     : "GuestResponse",
  "requestMessageID": 31,
  "timestamp"       : "2026-08-05T14:35:24.568021+05:30"
}

```


Code	Meaning / condition
40009	Checkin is not specified
40011	Checkin is already checked out
50001–50015	Per-guest checks, as Section 1.2
50009	Relationship code missing or invalid — required for every non-main guest
50010	Guest is not specified
50014 / 50015	Guest check-in before the stay's, or in the future
50016	That document number is already on this stay
60001–60013	Per-attachment checks, as Section 1.2

Warning Two defects affect this endpoint. It does not check whether the stay has been cancelled, so a guest can be added to a cancelled check-in. And the shared field-format validator does not run here, so name, Arabic name, email, mobile, document-number and birth-place formats are not enforced. Validate these client-side. See Appendix B, items 1 and 2.

4.3 Guest PUT — amend a guest

Updates a guest already on the stay. Matching is on DocumentNumber: a guest whose document number already exists is updated in place rather than duplicated.

4.3.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "Comment": "payload collection - edit an escort already on the checkin",
  "MessageType": "GuestUpdateRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-edit-escort-001",
  "Timestamp": "2026-08-04T08:45:00",
  "GuestInfo": {
    "UID": null,
    "GuestCode": "PC-ESCORT-0500",
    "FirstName": "Alan Mathison",
    "LastName": "Turing",
    "ArabicName": "آلان تورينج",
    "GenderCode": "male",
    "Email": "alan.m.turing@example.com",
    "Mobile": "501550099",
    "ResidenceCountryPhone": "971501550099",
    "BirthDate": "1990-11-20T00:00:00",
    "BirthPlace": "Manchester",
    "NationalityCode": "GB",
    "ResidenceCountryCode": "AE",
    "RelationshipCode": "familyMember",
    "VisitPurposeCode": "tourism",
    "CheckinDateTime": "2026-08-04T08:30:00",
    "CheckoutDateTime": null,
    "IsMainGuest": false,
    "AttachmentTypeCode": "passport",
    "DocumentNumber": "GB1122334",
    "IssueCountryCode": "GB",
    "IssueDate": "2019-05-01T00:00:00",
    "ExpiryDate": "2029-05-01T00:00:00",
    "EmirateCode": null,
    "RequiresAccessibility": false,
    "AccessibilityTypes": [],
    "OtherAccessibilityType": "",
    "Attachments": [
      {
        "Id": 0,
        "UID": null,
        "AttachmentCode": "PC-D0C-0500",
```

```

      "Name": "Image_1.jpg",
      "Size": 160,
      "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
    }
  ]
}

```

4.3.2 Request

Method	URL
PUT	/guest

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The check-in the guest belongs to.
GuestInfo	GuestInfo	Yes	Full object, not a delta.
Comment	string	No	Free-text remarks.

4.3.3 Response

200 — as 4.2.3, with messageType "GuestResponse".

Code	Meaning / condition
40009	Checkin is not specified
40010	Checkin is already cancelled
40011	Checkin is already checked out
50001–50015	Per-guest checks
60001–60013	Per-attachment checks

Note Unlike 4.2, this endpoint deliberately does not raise 50016 for a document number already on the stay — that is what makes it an amendment rather than an addition.

5. Guest Checkout

5.1 GuestCheckout POST

Checks one guest out while the stay continues. This is also the only mechanism for transferring the main-guest role.

5.1.1 Request body sample — an escort departs

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "GuestUID": "REPLACE_WITH_GUEST_ID",
  "CheckoutDateTime": "2026-08-04T10:00:00",
  "NewMainGuestUID": null,
  "MessageType": "GuestCheckoutRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-guestcheckout-001",
  "Timestamp": "2026-08-04T10:00:00"
}
```

5.1.2 Request

Method	URL
POST	/guestcheckout

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The stay.
GuestUID	string	Yes	The guest id of the departing guest — from GET /guest.
CheckoutDateTime	DateTime	Yes	Departure date and time.
NewMainGuestUID	string	Conditional	The guest id of the successor. Required in practice whenever the departing guest is the main guest.

5.1.3 Main-guest handover

When the main guest leaves but the stay continues, set NewMainGuestUID to the guest id of a guest who is staying:

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "GuestUID": "REPLACE_WITH_DEPARTING_MAIN_GUEST_ID",
  "CheckoutDateTime": "2026-08-04T10:15:00",
  "NewMainGuestUID": "REPLACE_WITH_REMAINING_ESCORT_GUEST_ID",
  "MessageType": "GuestCheckoutRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-mainguest-handover-001",
  "Timestamp": "2026-08-04T10:15:00"
}
```

The API then performs, in order:

Step	Effect
1	Writes a main-guest-change record and a log entry
2	Clears IsMainGuest on the outgoing guest and copies the successor's relationship onto them
3	Sets IsMainGuest on the successor and clears their relationship
4	Writes a new version snapshot for both guests

Step	Effect
5	Checks the departing guest out

Warning There is no separate main-guest-change endpoint; the handover is driven entirely by NewMainGuestUID. If the main guest is checked out without a successor this build does not refuse the request — it leaves the stay with no main guest, which breaks subsequent version snapshots. The 1.ID specification defines error 50018 for exactly this case; it is not implemented. Always send a successor.

5.1.4 Response

200 — the envelope with the updated stay.

Code	Meaning / condition
40004	Check-out date must not be in future
40009	Checkin is not specified
40010	Checkin is already cancelled
40011	Checkin is already checked out
40014	Checkout date time is invalid
40032	Check-out must be at or after that guest's check-in
50017	Guest is not in the room — already checked out

6. Guest Attachment

6.1 GuestAttachment POST

Adds further documents for a guest already on an open stay. This endpoint writes to the filesystem as well as the database.

6.1.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "GuestUID": "REPLACE_WITH_GUEST_ID",
  "MessageType": "GuestAttachmentRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-attachment-001",
  "Timestamp": "2026-08-04T09:00:00",
  "Attachments": [
    {
      "Id": 0,
      "UID": null,
      "AttachmentCode": "PC-DOC-EXTRA-0600",
      "Name": "Image_2.jpg",
      "Size": 160,
      "ContentBase64Encoded": "/9j/4AAQSkZJRgABAQEAYABgAAD/2wBDAAgGBgcG..."
    }
  ]
}
```

6.1.2 Request

Method	URL
POST	/guestattachment

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The stay.
GuestUID	string	Yes	The guest id — from GET /guest, not from POST /checkin.
Attachments	IList<AttachmentInfo>	Yes	At least one, each under 200 KB decoded.

6.1.3 Response

200 — the envelope with the guest and their stored documents.

Code	Meaning / condition
40009	Checkin is not specified
40010	Checkin is already cancelled
40011	Checkin is already checked out
50010	Guest is not specified — no GuestUID supplied
60001	Attachment is not specified
60004	Attachment should not exceed 200 KB in size

Warning There is no check that the content is actually an image. The 1.ID specification defines 60005 for this; it is not implemented, so a non-image payload is accepted and then fails inside the image resizer as an HTTP 500.

Note Files are written under BaseDirectoryPath\Attachments\...\<GuestId>. This endpoint de-duplicates filenames: a second Image_2.jpg is stored as Image_2_1.jpg.

7. Room Change

7.1 RoomChange POST

Moves an open stay to a different room: frees the old room, flags the new one, and rewrites the room number on the check-in.

7.1.1 Request body sample

```
{
  "CheckinUID": "REPLACE_WITH_CHECKIN_UID",
  "NewRoomNumber": "004",
  "EffectiveDate": "2026-08-04T09:30:00",
  "Comment": "payload collection - move the stay to another room",
  "MessageType": "RoomChangeRequest",
  "ClientUID": "Establishment101",
  "MessageUID": null,
  "CorrelationUID": "test-roomchange-001",
  "Timestamp": "2026-08-04T09:30:00"
}
```

7.1.2 Request

Method	URL
POST	/roomchange

Parameter	Type	Required	Notes
CheckinUID	string	Yes	The stay to move.
NewRoomNumber	string	Yes	Must exist and be unoccupied.
EffectiveDate	DateTime	Yes	When the move takes effect. Note the field name — not EffectiveDateTime.
Comment	string	No	Free-text remarks.

7.1.3 Ordering rules

EffectiveDate must be after	Reported as
The stay's check-in date and time	40004
Any previous room change	free text
Any main-guest change	free text
The most recently added escort's check-in and check-out	free text

7.1.4 Response

```
{ "CheckinUID": 17 }
```

Code	Meaning / condition
40004	Room change date in the future, or before the stay's check-in
40009	Checkin is not specified
40010	Checkin is already cancelled
40011	Checkin is already checked out

Code	Meaning / condition
80001	The new room does not exist
80003	The new room is occupied

8. Reference lookup endpoints

Note These endpoints are an extension of this build. The 1.ID specification distributes the same data as a shared spreadsheet.

Each returns the whole table as a JSON array. No body, no parameters. Read them at start-up to populate your dropdowns rather than hard-coding values.

Endpoint	Rows	Feeds
GET /country	252	NationalityCode, ResidenceCountryCode, IssueCountryCode
GET /emirate	9	EmirateCode
GET /room	31	RoomNumber — IsChecked true means occupied
GET /documenttype	21	AttachmentTypeCode
GET /visitpurpose	18	VisitPurposeCode
GET /relationship	5	RelationshipCode
GET /paymenttype	2	PaymentMethodCode
GET /cardtype	5	CreditCardTypeCode
GET /checkintype	2	Reference
GET /checkouttype	2	1 = check-out, 2 = cancellation
GET /escorttype	2	1 ChildEscort, 2 AdultEscort — see Appendix B item 10
GET /accessibilitytype	4	AccessibilityTypes[].Code
GET /cancellationreason	2	CancellationReasonCode
GET /dtcmaction	18	Reference

Warning A 200 from these endpoints is not proof of success. They catch their own database errors and return {"error": "..."} with status 200. Inspect the body, not just the status code.

Diagnostic table dumps

Thirteen further read-only endpoints dump raw tables, useful for confirming what a write actually did: /checkintable, /guesttable, /checkinguest, /guestversion, /guestattachment, /guestdocumentimage, /payment, /log, /checkout, /guestcheckout, /roomchange, /checkincancellation, /mainguestchange. The same 200-with-error caveat applies.

9. Monitoring endpoints

Note These endpoints are an extension of this build and are not in the 1.ID specification. They are not present in older deployments — check GET /routes before relying on them.

Endpoint	Purpose
GET /health	Full readiness check — database, storage, configuration. 200 when healthy, 503 with the failing check named. Start here.
GET /health?deep=false	Skips the database and filesystem probes. Use for a fast poll.
GET /ping	Cheapest liveness check. Returns "pong".
GET /version	Version, process id, start time.
GET /metrics	Request counters and recent traffic.
POST /metrics/reset	Zeroes the counters. Empty body.
GET /routes	Every URL this build serves — authoritative when this document and the software disagree.
GET /status	HTML dashboard. Open in a browser.

Appendix A. Deviations from the 1.ID specification

Every item below was verified against the source of build v1.0.0. This appendix exists so that a client written from the 1.ID specification alone does not fail in production.

A.1 Error codes defined but not implemented

The specification defines 71 error codes. This build implements 49. The 22 below are never returned:

Code	Specification text	What happens instead
40001	Checkin wrong Establishment UID	ClientUID is never validated
40012	Payment order generated, cannot modify	No payment-order concept in this build
40015	Check-in overlaps another check-in, same room	Free-text message
40016	Check-out overlaps another check-in, same room	Free-text message
40019	Check-in overlaps a room change	Free-text message
40020	Check-in overlaps a main-guest change	Free-text message
40021	Check-out overlaps a room change	Free-text message
40022	Check-out overlaps a main-guest change	Free-text message
40024	Payment order for that month and year	Not implemented
40026	Check-in before establishment opening date	No establishment profile
40027	Waiting-for-room limit reached	Not implemented
40030	Establishment profile incomplete	Not implemented
50012	Arabic name is required	Enforced on the check-in endpoints, but reported as a field-keyed message rather than this code
50018	Guest is a main guest	Not implemented — see Section 5.1.3
50019	Guest is a visitor	Not implemented
50020	Main-guest change outside the stay window	Free-text message
50021	Guest does not exist	Not implemented; 50010 covers the missing-identifier case
50022	Guest code already exists	Not implemented — duplicates are accepted
60002	Attachment is not specified	Not implemented; 60001 covers it
60003	Attachment is not specified	Not implemented; 60001 covers it
60005	Attachment should be an image	Not implemented — see Section 6.1
80004	Room change outside the stay window	Split between code 40004 and free text

Important Eight of these — 40015, 40016, 40019, 40020, 40021, 40022, 50020 and 80004 — are the overlap rules. The behaviour is implemented; only the numbering is missing. Your error handling must accept a free-text sentence as well as a numbered code.

A.2 Codes returned that the specification does not define

Code	Text	Note
40050	Checkin room is occupied	The specification folds this into 40003. This build keeps 40003 for an inactive room and raises 40050 for an occupied one. Handle both.

A.3 Codes carrying more than one meaning

Six codes are reused for unrelated conditions. Match on the message text, not the number alone.

Code	Specification meaning	Also returned for
40004	Date must not be in the future	Room change earlier than the stay's check-in
40035	House use does not require a payment method	A guest window falling outside the stay, on AddBackdatedCheckin
50001	Guest invalid first name	Guest invalid guest code
50014	Guest check-in before the stay's check-in	Three further ordering failures on AddBackdatedCheckin
60008	Attachment issue date cannot be in the future	Missing "other" accessibility description
60011	Attachment expiry date is not specified	The document is expired

A.4 Validation rules that differ

Rule	Specification	This build
EmirateCode	Required if the document type is Emirates ID	Required whenever IssueCountryCode is AE, whatever the document type. A UAE-issued passport therefore needs one; an Emirates ID issued elsewhere does not.
Email	Optional	The key must be present. It may be null, but omitting it entirely produces an HTTP 500.
Attachments on Guest POST/PUT	The sample sends null	At least one attachment is required (60001).
Cancellation window	Not mentioned	The cancellation must fall in the same calendar month as the check-in.

A.5 Other differences

Aspect	Specification	This build
URL prefix	api/checkin/	/checkin — no api/ prefix
Attachment uid in responses	A GUID string	An integer
CheckinUID in bare responses	A string, "78"	A number, 78
GuestAttachment response envelope	PascalCase keys	camelCase, consistent with every other endpoint
Guest sample values	Randomised placeholders such as "2HELQSXB"	Rejected by the name and date validators — use the samples in this document

Appendix B. Known issues

Defects in build v1.0.0, ordered by their likely impact on an integration. These are disclosed so that you can design around them.

#	Severity	Issue	What you should do
1	High	The shared field-format validator does not run on POST /guest or PUT /guest — it reads the wrong key. Guests added after check-in bypass name, Arabic name, email, mobile, document-number and birth-place format checks.	Validate these fields client-side. Do not treat acceptance as proof the data is well formed.
2	High	POST /guest does not check whether the stay has been cancelled, so a guest can be added to a cancelled check-in. PUT /guest does check.	Check the stay state before adding a guest.
3	High	PUT /checkin locates the guest row to update using the guest's newest record across all check-ins, so a repeat visitor on a newer stay can have the wrong stay amended.	Avoid amending a stay for a guest who has since checked in elsewhere.
4	High	Error 50018 is not implemented: the main guest can be checked out with no successor, leaving the stay without a main guest and breaking later version snapshots.	Always send NewMainGuestUID when the departing guest is the main guest.
5	High	POST /checkin is not transactional. It commits payment, check-in, guest, attachment, join row, log and version separately, so a failure part-way leaves orphan records.	Treat an HTTP 500 as "state unknown". Query /checkintable before retrying.
6	Medium	Seven guest fields are read without a default and raise HTTP 500, not 400, when absent.	Send every key, using null where you have no value.
7	Medium	Reference lookups and table dumps return HTTP 200 with an {"error": ...} body when the database call fails.	Inspect the body, not the status code.
8	Medium	No content-type check on attachments (60005 unimplemented); a non-image is accepted and then fails as a 500 in the resizer.	Verify the content is a JPEG before sending.
9	Medium	Duplicate guest codes are accepted (50022 unimplemented).	Enforce uniqueness client-side.
10	Medium	EscortTypeId is fixed at 2 (AdultEscort) in every write path. ChildEscort cannot be recorded and the escort count fields are never populated.	Raise this with us if you need to distinguish child from adult escorts.
11	Low	The timestamp field in responses is computed once at process start, not per request.	Do not use it to order events; use your own CorrelationUID.
12	Low	Visa Electron in the card type table has the DtcCode "0".	Do not offer it until the data is corrected.

Appendix C. Setting up a test environment

C.1 Prerequisites

Component	Requirement
Operating system	Windows 10, 11, or Server 2019 and later
SQL Server	2019 or later, with the ITSthe1.ID database restored
ODBC driver	ODBC Driver 17 for SQL Server — named explicitly in the connection string
Python	3.11 or later, only if running from source. The packaged executable needs nothing installed.
Storage	A writable folder for guest document images

C.2 Configuration

Copy config.example.json to config.json, placing it beside run.py when running from source or beside run.exe when deployed, and complete the connection string:

```
{
  "SQLALCHEMY_DATABASE_URI":
    "mssql+pyodbc://USER:PASSWORD@HOST\\INSTANCE/itsthe1.id?driver=ODBC+Driver+17+for+SQL+Server",
  "SCHEMA_NAME"      : "[itsthe1.id].[itsthe1.id]",
  "BaseDirectoryPath" : "C:/ITSthe1/WPF/DOTS_Storage",
  "HOST"              : "127.0.0.1",
  "PORT"              : 5030
}
```

Warning Write paths with forward slashes. JSON treats a backslash as an escape character, and a pasted Windows path such as C:\build\files parses silently into something else entirely — the only symptom is a storage folder that "does not exist". SCHEMA_NAME is a [database].[schema] prefix applied to every table; if it is wrong, every endpoint fails with "Invalid object name".

The file is read once at start-up. Restart the service after editing it.

C.3 Starting the service and confirming it works

Step	Action	Expected
1	Run dist\run.exe, or python run.py from source	A tray icon appears; green means every health check passed
2	GET /health	200 with healthy true. A 503 names the failing check
3	Run docs\payloads\check_all_reads.ps1	OK on every read-only endpoint. Writes nothing
4	Import the Postman collection and environment	61 requests in 8 folders
5	Set baseUrl, then run folder 1	All monitoring endpoints green

C.4 Accompanying files

This document is issued with a bundle containing:

Path	Contents
docs/payloads/*.json	The seventeen payloads reproduced in this document, ready to send
docs/payloads/ITSthe1.ID.postman_collection.json	Every endpoint as a Postman request, with assertions and automatic identifier capture
docs/payloads/ITSthe1.ID.postman_environment.json	The matching environment
docs/payloads/check_all_reads.ps1	Read-only smoke test — writes nothing
docs/payloads/run_write_flow.ps1	Drives the full lifecycle end to end. Writes to the database
config.example.json	Configuration template

C.5 A working call sequence

Each step supplies an identifier the next one needs. Use a room reported as free by GET /room so that live stays are never disturbed.

#	Call	Take from the response
1	GET /health	Confirm healthy before anything else
2	The reference lookups	Code values; also proves SCHEMA_NAME is correct
3	POST /checkin on a free room	CheckinUID
4	GET /checkin	Confirm the stay was created
5	POST /guest	Adds an escort
6	GET /guest	The guest ids for steps 7–9
7	POST /guestattachment	A second document for that guest
8	POST /roomchange	Move to another free room
9	POST /guestcheckout	The escort departs
10	POST /checkout	Close the stay

Important Timestamps must increase across the sequence. Room change and check-out are both validated against the most recently added guest, so order them: check-in < escort check-in < room change < escort check-out < check-out. The supplied payloads are already ordered this way.

Appendix D. Security and data handling

The API has no authentication. Any client that can reach the port can create check-ins and read guest data. It is designed to run on a closed hotel network.

Control	Guidance
Binding	Keep HOST as 127.0.0.1 when the client runs on the same machine.
Firewall	If the client is on another machine, restrict port 5030 to specific client addresses.
Internet exposure	Never expose this port to the internet.
Credentials	The database password is stored in plain text in config.json. Protect it with file-system permissions. The API masks it wherever it reports its own configuration.

Important Guest records contain personal data — names, passport and Emirates ID numbers, and photographs of identity documents — held both on disk under BaseDirectoryPath and as blobs in the database. Any database copy or storage folder shared for testing must be treated as regulated data. Use synthetic guests: every payload in this document does.

Support

When reporting a failure, quote the CorrelationUID you sent. It is echoed back unchanged and is the fastest way to locate the request in the service log. Logs are written to logs\api.log, rotating at 5 MB with five files retained; set LOG_LEVEL to DEBUG and restart for full request bodies.

Document status. Every statement in this specification was verified against the source of build v1.0.0. The read-only endpoints were additionally exercised against a live instance. The request and response shapes for the write endpoints are derived from the source rather than from observed responses, because the reference database used for verification is missing a table required by the document-storage path; that is a database gap, not a defect in the code.